

Assignment 3: Diffusion Playground

Sampling, Editing, and Illusions with a Pretrained
Text-to-Image Diffusion Model (DeepFloyd IF)

Omer Yosef (207003187)

Gal Barak (211699707)

Neural Graphics (3683-8901)

Note on images. All figures are the outputs actually generated by `Assignment3.ipynb` (the submitted notebook) and are stored in `./writeup_images/`. Results are stochastic and prompt-dependent; the exact prompts used are listed under each task and summarised in the appendix.

Part 0: Loading the Model

Task 0.1: Sampling with your own prompts + inference-step comparison

Prompts used (all exact keys in the precomputed embedding dictionary):

- "an oil painting of an old man"
- "an oil painting of people around a campfire"
- "a rocket ship"

Random seed: 100 (set with `seed_everything(100)`, and reused for all later parts).

Stage 1 (64×64):



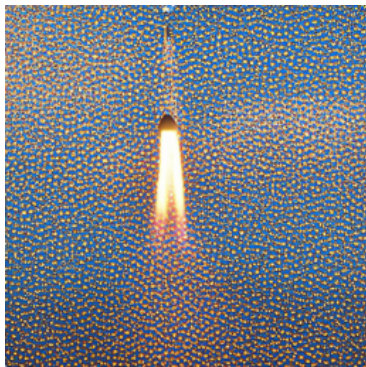
old man / campfire / rocket ship — stage 1

Stage 2 (upsampled to 256×256):



old man / campfire / rocket ship — stage 2

Low vs. high `num_inference_steps` (prompt "a rocket ship"):



5 steps (left) vs. 50 steps (right)

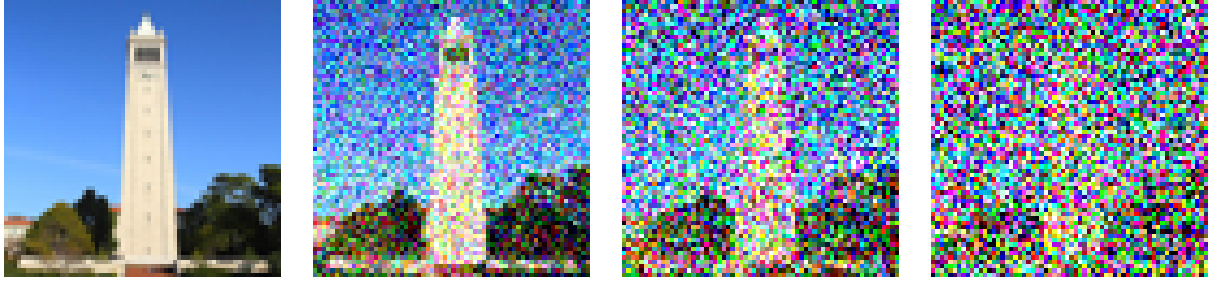
Discussion.

- *Stage 1 vs. Stage 2.* Stage 1 sets the overall composition and content at low resolution (64×64); it looks right but soft. Stage 2 is a conditional super-resolution diffusion model that adds high-frequency detail (edges, texture) to reach 256×256 . The content stays the same, but the sharpness and fine structure improve a lot.
- *5 vs. 50 steps.* With only 5 denoising steps the sampler can't remove the noise accurately, so the image is blurry, low-contrast, and hard to read. At 50 steps each reverse step is a smaller, more accurate update, which gives a sharp, coherent image. Most of the improvement happens by about 20 steps and then levels off; the 50-step result is only a little better than a 20-step one despite costing much more.

Part 1: Sampling-Loop Mechanics

Task 1.1: The Forward Process

The forward process adds noise per the pretrained schedule: $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$. Test image noised at $t \in \{250, 500, 750\}$:



original / $t = 250$ / $t = 500$ / $t = 750$

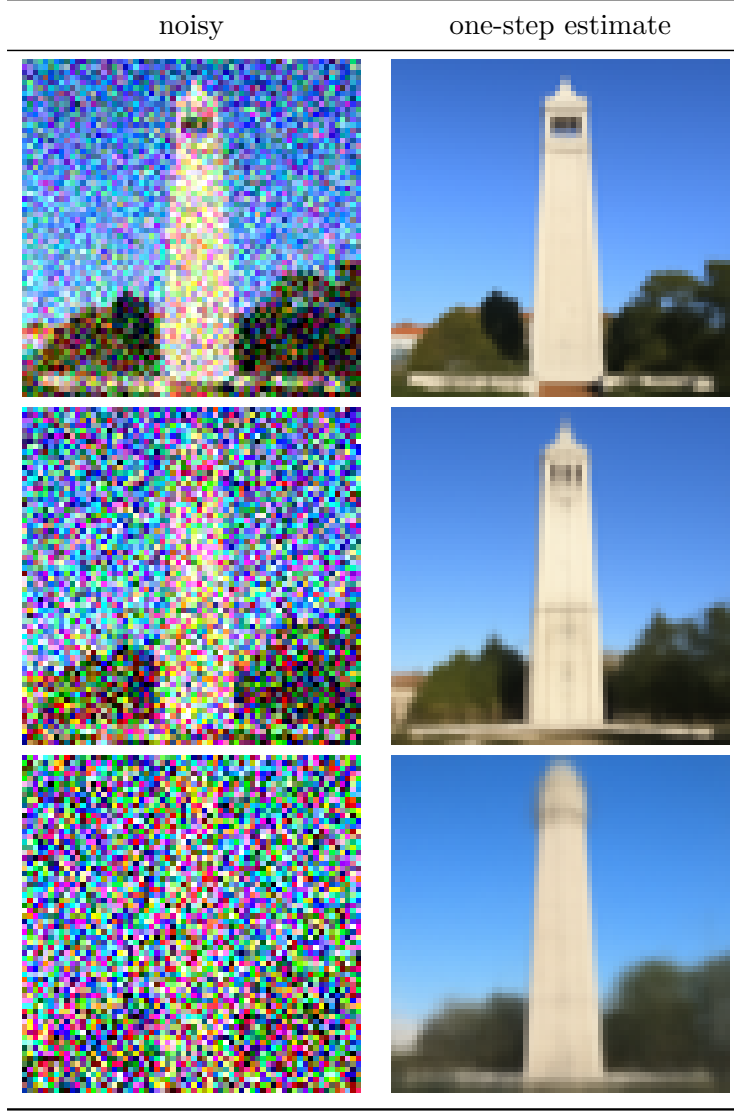
Discussion. As t grows, $\sqrt{\bar{\alpha}_t}$ shrinks and $\sqrt{1 - \bar{\alpha}_t}$ grows, so the clean signal is progressively drowned out by Gaussian noise. At $t = 250$ the structure is still clearly visible; by $t = 750$ the image is almost pure noise.

Task 1.2a: One-Step Denoising

Using the UNet's noise estimate to jump straight to a clean-image estimate: $\hat{x}_0 = (x_t - \sqrt{1 - \bar{\alpha}_t} \hat{\epsilon}_\theta(x_t, t)) / \sqrt{\bar{\alpha}_t}$. Null prompt "a high quality photo", at $t \in \{250, 500, 750\}$:



original



$t = 250$ (top), $t = 500$ (middle), $t = 750$ (bottom)

Discussion. The one-step estimate is good at low noise ($t = 250$) but degrades badly as t rises: at $t = 750$ it is blurry and washed out, because the model must reconstruct the entire clean image from almost pure noise in a single shot.

Task 1.2b: Iterative Denoising (strided schedule)

`strided_timesteps = list(range(990, -1, -30))` → **34 timesteps** (990, 960, ..., 30, 0), following the DDPM reverse step

$$x_{t'} = \frac{\sqrt{\bar{\alpha}_{t'}} \beta_t}{1 - \bar{\alpha}_t} \hat{x}_0 + \frac{\sqrt{\alpha_t} (1 - \bar{\alpha}_{t'})}{1 - \bar{\alpha}_t} x_t + \sigma_t z,$$

with $\alpha_t = \bar{\alpha}_t / \bar{\alpha}_{t'}$, $\beta_t = 1 - \alpha_t$. The iterative loop was run starting from a test image noised to `i_start = 10`.

Task 1.2c: One-Step vs. Iterative (same noise level, `i_start = 10`)



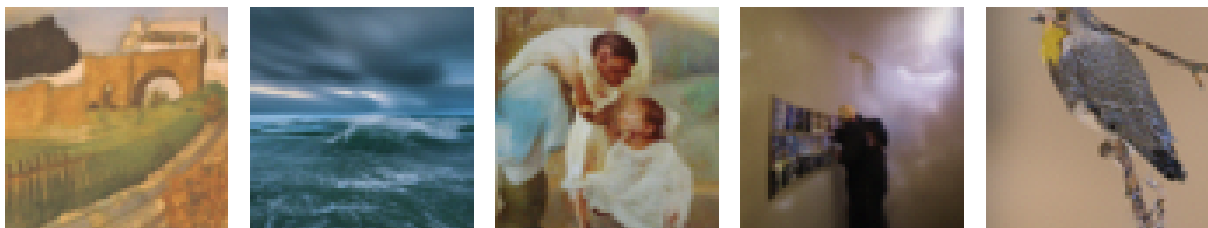
noisy (`i_start=10`) / iterative / one-step

Discussion: why iterative denoising beats one-step with the same model. Look at the rightmost panel above: the one-step result is a smeared, ghostly campanile, while the iterative one in the middle is crisp. Here is how we read it. At this noise level many different clean images could have produced the same noisy x_t , and the UNet only gets to output a single noise guess, so its one-shot answer lands on something like the average of all of them, which is exactly why `c31_2` looks washed out. The iterative version instead peels off a little noise per step; after each step the image is slightly cleaner, fewer clean images are still consistent with it, and the model can commit to one of them and start filling in real detail. Same weights in both runs, so all of the gain comes from splitting the work into many small, easy steps rather than one big leap.

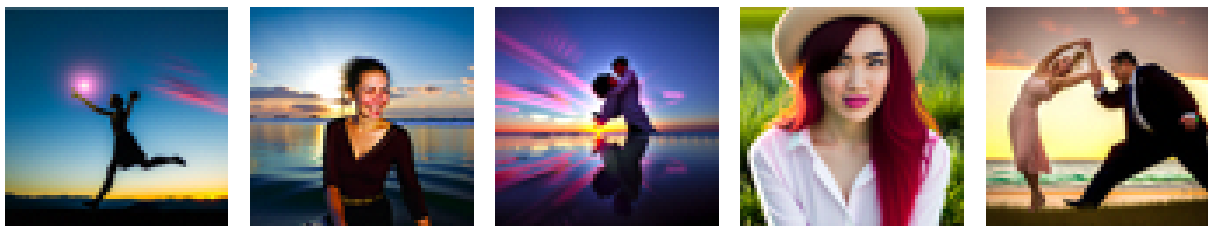
Task 1.3: Sampling With and Without CFG

Prompt "a high quality photo"; CFG uses $\varepsilon_{\text{cfg}} = \varepsilon_u + \gamma(\varepsilon_c - \varepsilon_u)$ with $\gamma = 7$.

Without CFG (5 samples):



With CFG, $\gamma = 7$ (5 samples):



Discussion. The top row (no CFG) is pretty underwhelming: dull, low-contrast, and a couple of them barely look like photos at all. The bottom row with $\gamma = 7$ is a clear step up: sharper,

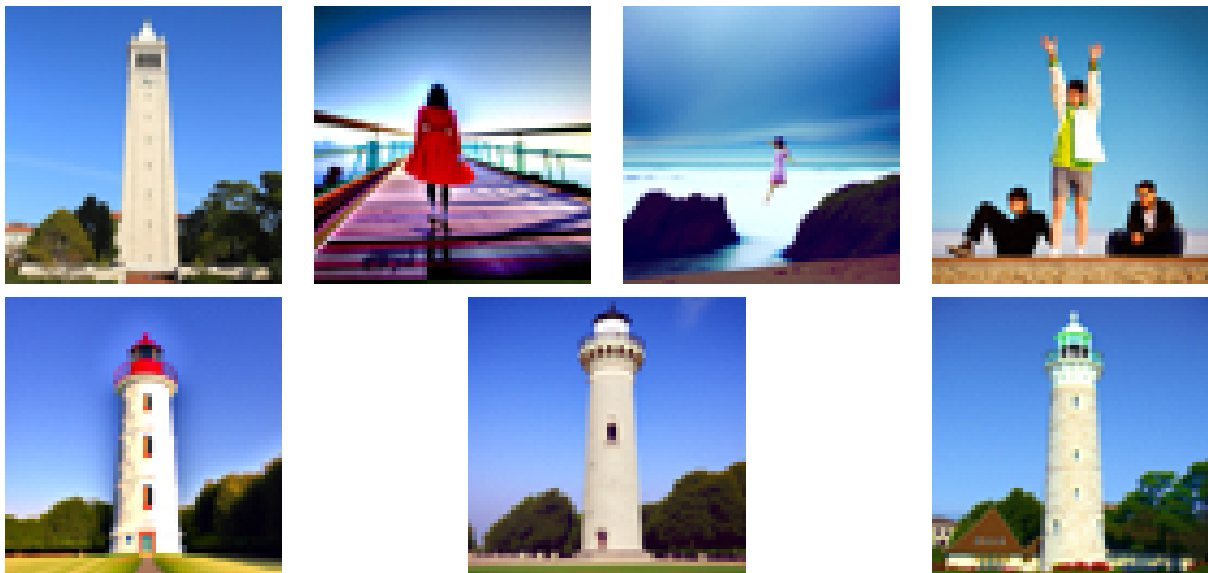
more saturated, and much more convincingly photographic, which makes sense because guidance pushes away from the unconditional estimate and leans harder into the conditional one. The cost shows up in variety: our five CFG samples resemble each other more than the five un-guided ones do.

Part 2: Image Editing

Task 2.1: SDEdit on the provided image and on your own image

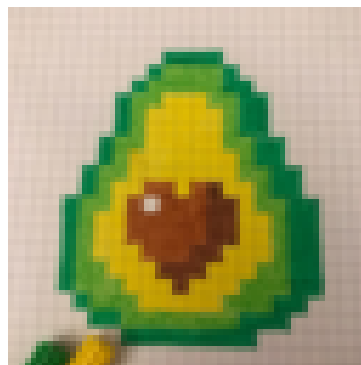
Procedure: noise the input to the timestep for each $i_start \in \{1, 3, 5, 7, 10, 20\}$, then run `iterative_denoise_cfg` from that point with the null prompt "a high quality photo".

Provided test image (campanile):

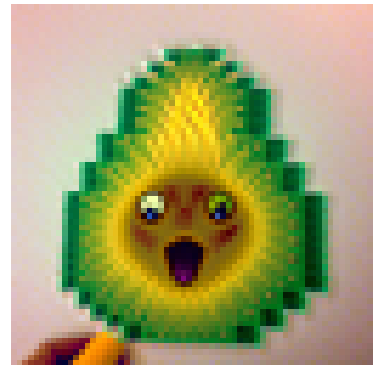
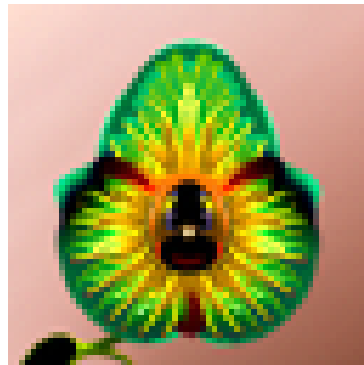
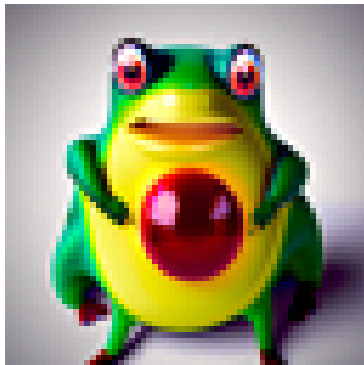


original / $i_start = 1, 3, 5, 7, 10, 20$

Own image, web image (source: a downloaded web image):

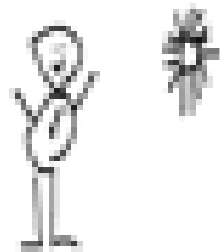


web source

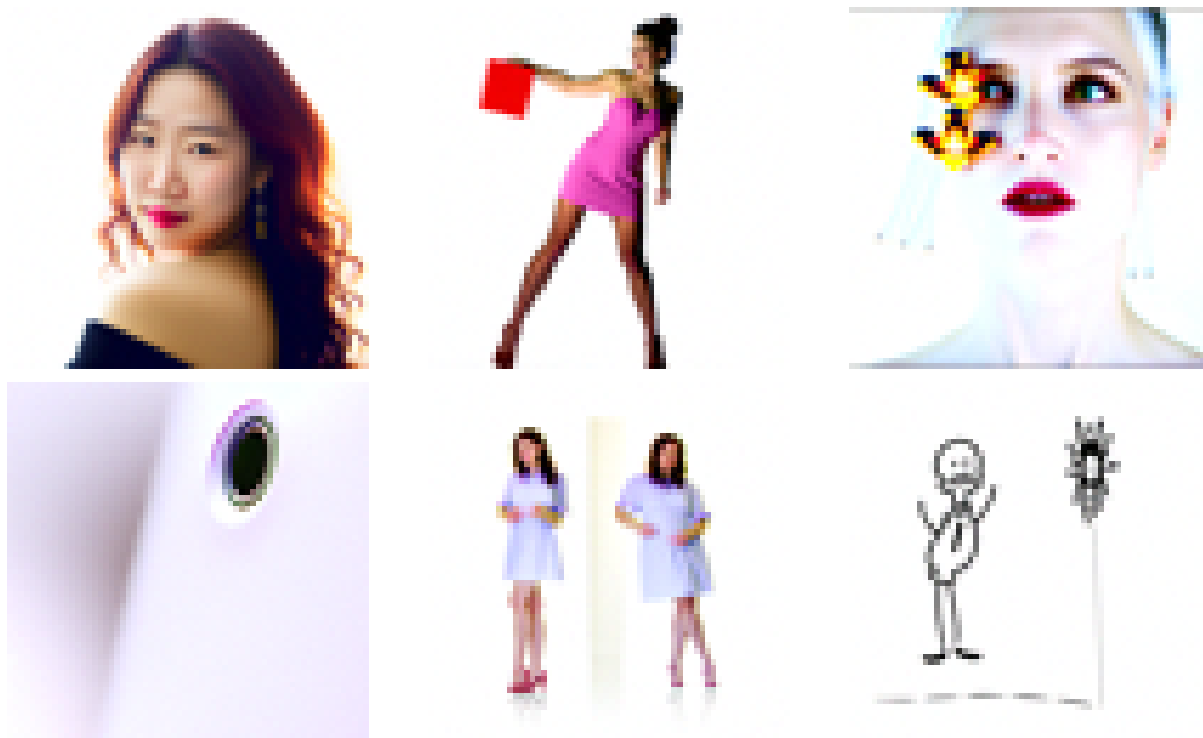


$i_start = 1, 3, 5, 7, 10, 20$

Own image, hand-drawn:



hand-drawn source



$i_start = 1, 3, 5, 7, 10, 20$

Discussion: edit strength vs. i_start . i_start controls how much noise we add before denoising again, i.e. how far back in the schedule we jump in. You can watch the effect march across the panels: at $i_start=1$ the output is almost the input untouched, since we barely disturbed it; by $i_start=5-7$ the model is clearly repainting things while the overall layout survives; and at $i_start=20$ so much of the original signal is gone that the model reinvents big chunks and the result wanders well away from what we fed it. So low i_start keeps you faithful and high i_start lets the model get creative. This was most fun to watch on the hand-drawn input, which only really starts to look like a real photo once we push i_start up to the higher values.

Task 2.2: Inpainting

New content is generated only inside a binary mask (1 = regenerate); everything outside is forced back to the original image, re-noised to the current timestep, at every step.

Provided test image (mask over a region):

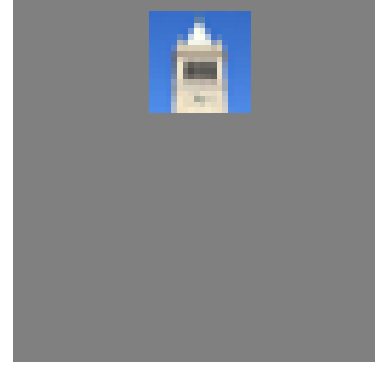
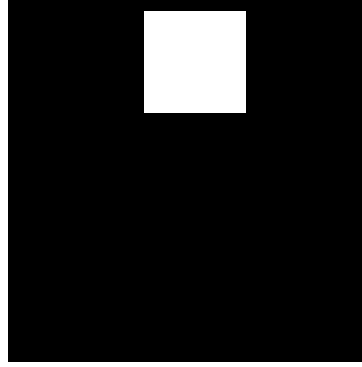
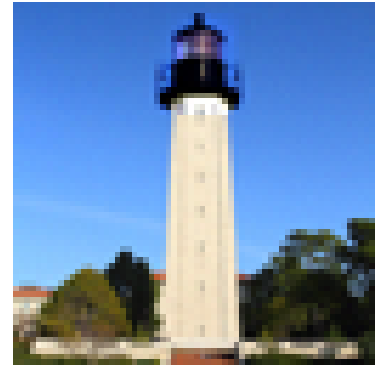
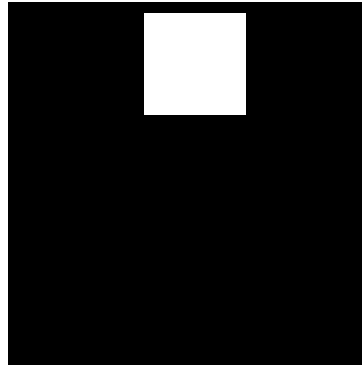
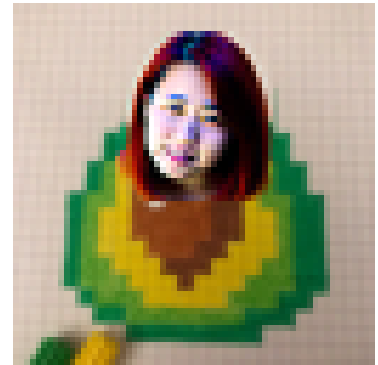
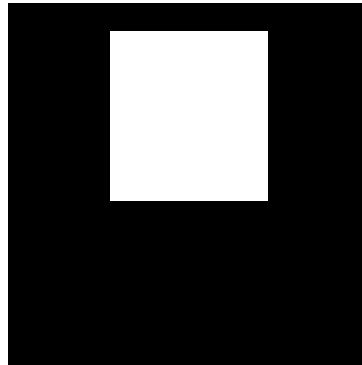
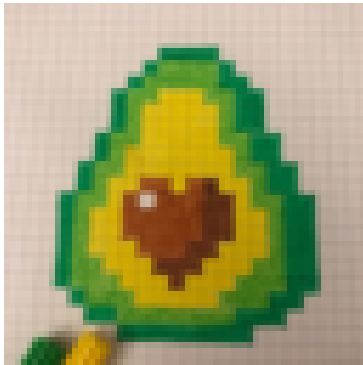


image / mask / to replace



original / mask / inpainted

Own image, own mask (inpainting the web image with a custom rectangular mask):

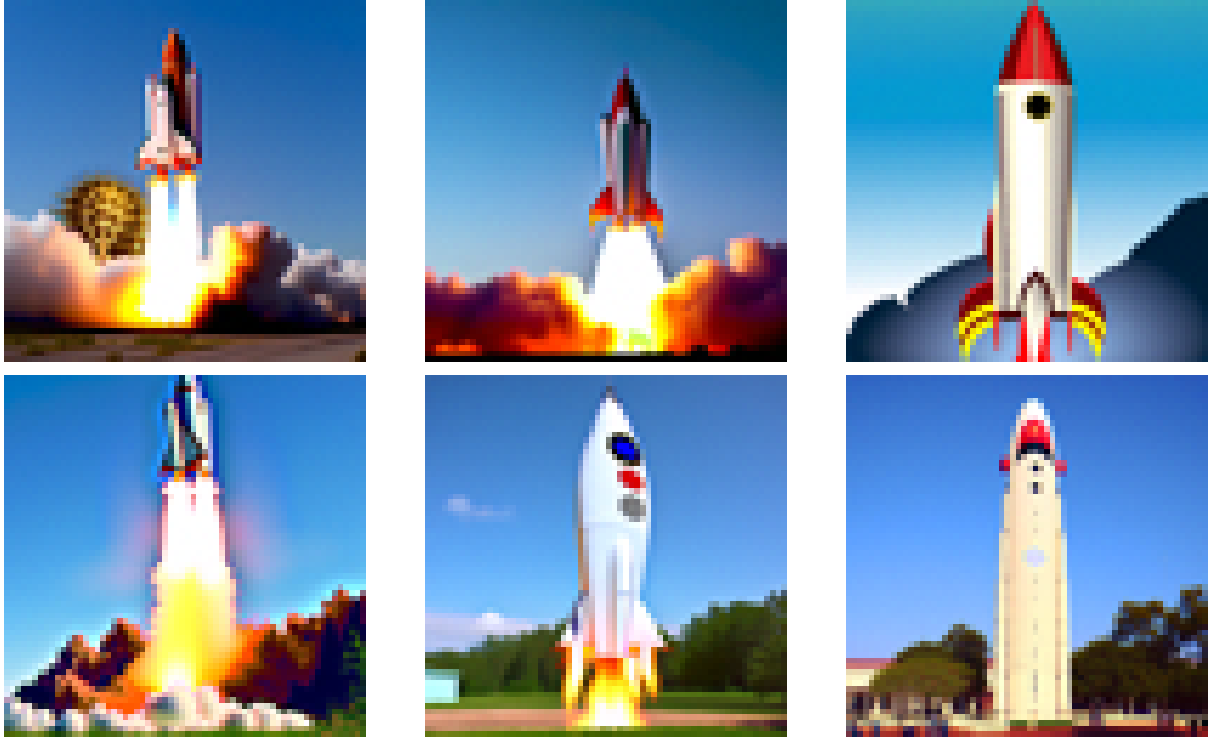


original / mask / inpainted

Discussion. The reverse process runs normally inside the mask, but at each step the region outside the mask is overwritten with the original image noised to match the current timestep. This keeps the surroundings fixed while letting the model generate new, context-consistent content inside the masked region. The boundary stays coherent because the model always sees the true (re-noised) surroundings while it denoises.

Task 2.3: Text-Conditioned Image-to-Image Translation

Same as SDEdit but steered by a real prompt instead of the null prompt. **Prompt used:** "a rocket ship", $i_start \in \{1, 3, 5, 7, 10, 20\}$ on the provided test image.



$i_start = 1, 3, 5, 7, 10, 20$

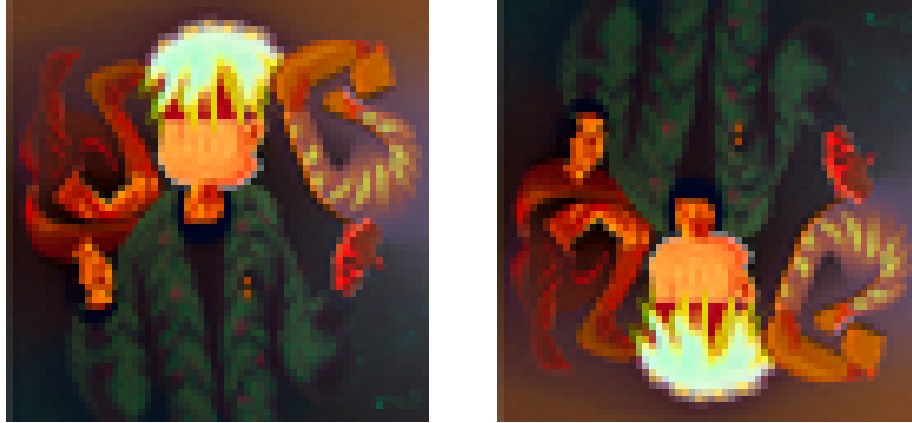
Discussion. Compared with the null-prompt edit in Task 2.1, the text prompt injects *semantic* content, not just “realism.” At low i_start the tall campanile silhouette is preserved but nudged toward rocket-like features; at high i_start the model is free enough to reshape the image into an actual rocket ship. The prompt controls which kind of real image the result is pulled toward, so the chosen words change what the edit becomes, not just how strong it is.

Part 3: Diffusion Illusions

Task 3.1: Visual Anagrams

Combined noise estimate $\varepsilon = \frac{1}{2}(\varepsilon_\theta(x_t, t, p_1) + \text{flip}(\varepsilon_\theta(\text{flip}(x_t), t, p_2)))$, $\gamma = 7$, denoised from pure noise ($i_start = 0$).

- p_1 (upright) = "an oil painting of an old man"
- p_2 (flipped) = "an oil painting of people around a campfire"



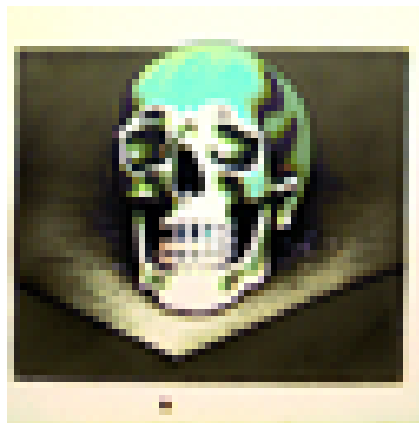
upright: an old man / flipped: people around a campfire

Discussion. Because the two noise estimates are averaged (one on the image, one on its vertical flip), the reverse process is simultaneously pulled toward both prompts under their respective orientations. The result reads as an old man right-side up and as people around a campfire when flipped.

Task 3.2: Hybrid Images (Bonus)

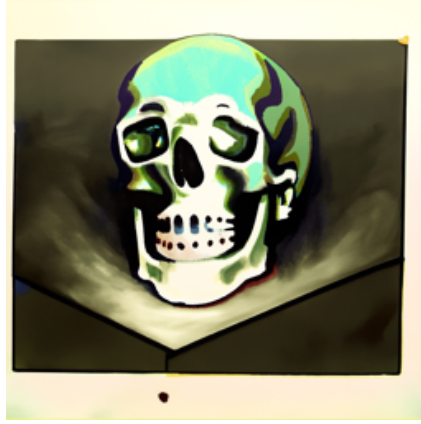
Combined estimate $\varepsilon = f_{\text{low}}(\varepsilon_{\theta}(x_t, t, p_{\text{low}})) + f_{\text{high}}(\varepsilon_{\theta}(x_t, t, p_{\text{high}}))$, where f_{low} is a Gaussian blur (kernel_size=9, sigma=2.0) and f_{high} is the estimate minus its own low-pass. $\gamma = 7$, i_start = 0.

- p_{low} (low frequencies / far) = "a lithograph of waterfalls"
- p_{high} (high frequencies / near) = "a lithograph of a skull"



hybrid (far: waterfalls / near: skull)

Upsampled to 256×256 (optional stage-2 pass):



hybrid upsampled

Discussion: which prompt dominates far vs. near, and why. The **waterfalls** prompt wins *from a distance*, because it provides the low spatial frequencies that set the overall shape and layout, and our eyes rely more on low frequencies when an image is small or blurry. The **skull** prompt wins *up close*, because it provides the high frequencies (fine texture and edges) that we only pick out at short range. Applying the low-pass/high-pass split to the *noise estimate* instead of to the pixels makes the model build this split directly into the generated image, so one picture reads as two different subjects depending on how far away you are. It works best when the two subjects have a similar rough outline, so the low-frequency prompt sets the shape and the high-frequency prompt just adds texture on top.

Appendix: Prompts and Parameters Used

Task	Prompt(s)	Key parameters
0.1	old man / campfire / rocket ship	stage 1 & 2 num_inference_steps=20; comparison 5 vs 50 (rocket)
1.1	n/a	$t \in \{250, 500, 750\}$
1.2a	"a high quality photo" (null)	$t \in \{250, 500, 750\}$
1.2b	"a high quality photo"	strided_timesteps = range(990,-1,-30) (34 steps), i_start=10
1.2c	"a high quality photo"	i_start=10 (one-step vs iterative)
1.3	"a high quality photo"	5 samples no-CFG + 5 samples CFG, $\gamma = 7$
2.1	"a high quality photo" (null)	i_start $\in \{1, 3, 5, 7, 10, 20\}$; campanile + web + hand-drawn
2.2	"a high quality photo"	provided mask (campanile) + custom mask on web image
2.3	"a rocket ship"	i_start $\in \{1, 3, 5, 7, 10, 20\}$
3.1	"an oil painting of an old man" / "an oil painting of people around a campfire"	$\gamma = 7$, i_start=0

Task	Prompt(s)	Key parameters
3.2	"a lithograph of waterfalls" (low) / "a lithograph of a skull" (high)	Gaussian blur k=9, $\sigma = 2.0$; $\gamma = 7$, i_start=0

CFG guidance scale used throughout Parts 2–3: $\gamma = 7$.